NetPay Manager: runbook de operación, rollback e incidentes

Objetivo: restablecer el servicio preservando la integridad de pagos, cargos y recibos.

1. Responsabilidades y criterios de intervención

El responsable de operación verifica infraestructura, servicios y certificados; el responsable de aplicación identifica la versión y compatibilidad de migraciones; el responsable de datos autoriza restauraciones y concilia movimientos. Una misma persona puede asumir las funciones, pero las decisiones y resultados se registran por separado.

Prioridad	**Ejemplo**	**Acción inicial**
Crítica	Posible pérdida o corrupción de pagos; escrituras inconsistentes	Suspender operaciones de escritura y conservar evidencias antes de restaurar
Alta	API inaccesible, base no disponible, certificado inválido	Delimitar infraestructura, TLS, aplicación y SQL
Media	Reporte falla, rol no permite una función, integración externa indisponible	Reproducir con usuario, filtro y componente específicos
Baja	Problema visual o advertencia sin interrupción	Registrar, priorizar y corregir mediante flujo de cambios

No hay un SLA contratado ni un RPO/RTO medido en la configuración actual. Los tiempos objetivo futuros se definen en el plan de escalabilidad; no se presentan como garantías operativas existentes.

2. Inventario de diagnóstico

Servicio	Instancia	Unidad	Directorio
Backend y PostgreSQL	`i-0e2bbaeb4cfd41f5d`	`netpay.service`	`/etc/netpay`
Frontend	`i-0172012226e3d165f`	`netpay-frontend.service`	`/etc/netpay-frontend`

Endpoints:
frontend `https://3.143.92.7/`;
backend `https://18.226.182.232/health`. Región: `us-east-2`.

2.1 Detener y volver a iniciar los servidores

Para una pausa planificada de una o dos semanas, se pueden detener las dos instancias desde AWS Console sin destruirlas. Las EIP están declaradas como recursos independientes y asociadas a las instancias; **detener/iniciar conserva las mismas IP mientras no se liberen o sustituyan esas EIP**. Los volúmenes EBS —incluidos la base y los archivos de configuración/certificados— también se conservan al detener las instancias.

Detención

1. Avisa la ventana de indisponibilidad, evita nuevos cobros/importaciones y espera a que terminen las operaciones en curso. Obtén y verifica una copia recuperable antes de la pausa; registra los SHA activos. Abre **AWS Console → EC2 → Instances** en `us-east-2`.
2. Identifica `i-0e2bbaeb4cfd41f5d` (backend/PostgreSQL) e `i-0172012226e3d165f` (frontend).
3. Selecciona ambas y elige **Instance state → Stop instance**. Confirma que la acción sea *Stop*, no **Terminate**.
4. Espera a que ambas figuren como **Stopped**. Durante la pausa no estarán disponibles la web ni la API; las operaciones no se procesan.

Arranque

1. En **EC2 → Instances**, selecciona el backend y usa **Instance state → Start instance**.
2. Espera a que figure **Running** y las comprobaciones de estado pasen. Comprueba que su EIP siga siendo `18.226.182.232` y valida `https://18.226.182.232/health`.
3. Inicia el frontend y espera a que figure **Running**. Comprueba que su EIP siga siendo `3.143.92.7` y abre `https://3.143.92.7/`.
4. Comprueba login, segundo paso si la cuenta lo usa, lectura de cartera y descarga de un comprobante existente. No crees un pago real solo para verificar disponibilidad.

5. Si las EIP cambiaron, se liberaron o no aparecen asociadas, detente y revisa **EC2 → Elastic IP addresses** y el estado Terraform antes de modificar DNS, CORS o configuración. No asumas que la IP temporal de la instancia es la EIP.

Detener una instancia evita su cargo de cómputo mientras permanece detenida, pero **no elimina todos los costos**: siguen aplicando almacenamiento EBS, Elastic IP/direcciones IPv4 públicas y otros recursos persistentes. Revisa precios/costos de AWS para la cuenta y región. No uses **Terminate**, `terraform destroy` ni elimines EIP/EBS: eso puede borrar recursos y comprometer datos. Tras una detención prolongada, verifica también certificados HTTPS y renovación automática.

Comprobaciones externas desde un equipo de operación:

curl --fail --show-error --max-time 20 https://18.226.182.232/health

curl --fail --show-error --max-time 20 -I https://3.143.92.7/

`/health` prueba API y `SELECT 1`. No comprueba correo, estado de cada tabla, exactitud de saldos ni integraciones de archivos. Un frontend con HTTP 200 tampoco prueba que el usuario pueda autenticarse.

3. Acceso administrativo y observabilidad

En AWS CLI:

El equipo de operación necesita AWS CLI v2, un perfil autorizado y vigente, `jq`, `curl` y OpenSSL. Los comandos de host se ejecutan en una sesión SSM de la instancia indicada; los de AWS se ejecutan en el equipo autorizado. Las restauraciones aisladas requieren además Docker y acceso al respaldo. No ejecutar los ejemplos de release con sus marcadores sin sustituir.

```
export AWS_PROFILE=netpay
export AWS_REGION=us-east-2
aws ssm describe-instance-information \
  --filters Key=InstanceIds,Values=i-0e2bbaeb4cfd41f5d,i-0172012226e3d165f \
  --query 'InstanceInformationList[].{Id:InstanceId,State:PingStatus}' --output table
```

Acceder por AWS Console → Systems Manager → Session Manager. El acceso CLI interactivo requiere el plugin Session Manager instalado y permisos administrativos correspondientes. El usuario de Actions solo necesita ejecutar los documentos de despliegue, no una sesión administrativa arbitraria.

3.1 Diagnóstico dentro de la EC2 backend

```
sudo systemctl status netpay.service --no-pager
sudo journalctl -u netpay.service --since '30 minutes ago' --no-pager
sudo docker compose --project-directory /etc/netpay \
  --env-file /etc/netpay/deploy.env -f /etc/netpay/compose.prod.yml ps
sudo docker compose --project-directory /etc/netpay \
  --env-file /etc/netpay/deploy.env -f /etc/netpay/compose.prod.yml logs --tail 100 app proxy db
sudo docker exec netpay-db-1 pg_isready -U netpay -d netpay_db
sudo docker exec netpay-app-1 curl --fail --silent http://localhost:8080/health
df -h
free -m
sudo docker stats --no-stream
```

La unidad es `Type=oneshot` con `RemainAfterExit=yes`: `active (exited)` puede ser normal después de un arranque exitoso. La disponibilidad de los procesos se verifica además con `docker compose ps`, healthchecks y endpoints. Los comandos anteriores presuponen los nombres de proyecto/contenedor generados por la configuración actual; verificarlos con `docker compose ps` antes de una restauración.

La unidad backend no define `ExecStop`: `systemctl stop netpay.service` no garantiza detener los contenedores. Para suspender escrituras durante recuperación, detener explícitamente proxy y API, manteniendo PostgreSQL para la copia/diagnóstico:

```
sudo docker compose --project-directory /etc/netpay \

  --env-file /etc/netpay/deploy.env -f /etc/netpay/compose.prod.yml stop proxy app
```

Registrar esta suspensión y evitar despliegues automáticos durante la intervención. La reactivación se realiza después de conciliar, mediante el arranque/reinicio controlado de la unidad y la verificación externa.

3.2 Diagnóstico dentro de la EC2 frontend

```
sudo systemctl status netpay-frontend.service --no-pager
sudo journalctl -u netpay-frontend.service --since '30 minutes ago' --no-pager
sudo docker compose --project-directory /etc/netpay-frontend \

  --env-file /etc/netpay-frontend/deploy.env -f /etc/netpay-frontend/compose.prod.yml ps
sudo docker compose --project-directory /etc/netpay-frontend \

  --env-file /etc/netpay-frontend/deploy.env -f /etc/netpay-frontend/compose.prod.yml logs --tail
100 frontend
```

No ejecutar `docker inspect` con salida completa ni imprimir `app.env`, `db.env` o SecureString en un registro compartido: pueden contener credenciales.

3.3 Monitoreo disponible y recomendado

Existen healthchecks Docker, estado de systemd, logs locales, resultados de SSM y workflows. EC2 ofrece métricas básicas como CPU y verificaciones de estado. No se declara CloudWatch Agent, centralización de logs, alertas de disponibilidad ni panel de métricas de memoria/disco.

Como rutina operativa propuesta, revisar disponibilidad pública y certificados antes de una jornada, consumo de disco y contenedores diariamente, errores de despliegue en cada liberación y capacidad/respaldos semanalmente. Automatizar estas revisiones requiere incorporar métricas, alarmas y un destino de notificaciones; no está resuelto por un healthcheck local.

3.4 Despliegue ordinario y cambios de configuración

1. Identificar repositorio y SHA aprobado; comprobar que la verificación CI terminó correctamente. Revisar cambios Flyway y compatibilidad con la imagen anterior. Si cambia el contrato, coordinar frontend y backend.
2. Confirmar respaldo recuperable, espacio disponible, SSM Online y acceso a ECR/Parameter Store. Registrar imagen anterior, digest y estado Flyway.
3. Para publicación ordinaria, usar el workflow del componente sobre `main`. Publica la imagen etiquetada con el SHA completo y ejecuta el documento SSM. Para una imagen ya publicada, usar la secuencia SSM de §4.2 con el SHA aprobado; no intentar sobrescribir un tag inmutable.
4. Esperar estado terminal de SSM y revisar contenedores. El backend ejecuta Flyway al arrancar; no hay una etapa que revierta esas migraciones si falla el binario.
5. Verificar HTTPS, CORS, login y una operación representativa; si aplica, segundo paso por correo y apertura XLS/PDF. Registrar ambos SHA y resultado. Solo declarar liberación correcta después de la comprobación funcional.

Actualizar Parameter Store por un canal administrativo autorizado y reiniciar `netpay.service` para regenerar el entorno y aplicar los valores. El escritor de configuración admite solo sus claves declaradas; `JWT_EXPIRATION` y `TWO_FACTOR_EXPIRATION_MINUTES` todavía no están en esa lista y requieren actualizar el mecanismo antes de incorporarlas al JSON. No imprimir el contenido del parámetro para diagnosticar.

Los valores de `environment.prod.ts` están compilados en Angular: cambiar API/dominio requiere reconstruir y publicar su imagen. Revisar también CORS y `FRONTEND_BASE_URL`; con rutas hash, comprobar que el enlace de recuperación recibido incluya `/#/reset-password`.

Rotación SQL: el valor `POSTGRES_PASSWORD` inicializa una base nueva; no cambia la contraseña de un volumen existente. Coordinar cambio de credencial dentro de PostgreSQL, SecureString y reinicio de API durante una ventana controlada. Una edición aislada del parámetro puede dejar la aplicación sin conexión.

4. Rollback de aplicación

4.1 Condiciones

Se utiliza una imagen anterior conocida y retenida en ECR. La política conserva hasta 12 imágenes, por lo que un SHA antiguo puede haber expirado. El rollback de backend requiere que el esquema aplicado siga siendo compatible con la versión anterior. Ante una migración destructiva o incompatible, detener el rollback automático y decidir corrección hacia adelante o restauración de datos.

Los scripts de release guardan el SHA anterior y, si falla el reinicio, intentan volver a ejecutarlo. Esta recuperación es de mejor esfuerzo: el workflow puede terminar fallido aunque el servicio anterior vuelva a estar disponible. Verificar el SHA real y el endpoint al cerrar el incidente.

4.2 Selección y ejecución desde el equipo autorizado

Ejemplo backend:

```
RELEASE_SHA='<SHA-anterior-validado-de-40-caracteres>'
INSTANCE_ID=i-0e2bbaeb4cfd41f5d
DOCUMENT=netpay-deploy
REPOSITORY=netpay-backend

aws ecr describe-images --repository-name "$REPOSITORY" \

  --image-ids "imageTag=$RELEASE_SHA" --query 'imageDetails[0].imageDigest' --output text
```

Continuar solo si ECR devuelve un digest y la revisión de compatibilidad es satisfactoria:

```
PARAMETERS=$(jq -n --arg release "$RELEASE_SHA" '{Release:[$release]}')
COMMAND_ID=$(aws ssm send-command --document-name "$DOCUMENT" \
  --instance-ids "$INSTANCE_ID" --parameters "$PARAMETERS" \
  --query Command.CommandId --output text)

aws ssm get-command-invocation --command-id "$COMMAND_ID" --instance-id
"$INSTANCE_ID" \
  --query '{Status:Status,Code:ResponseCode,Error:StandardErrorContent}' --output json
```

Para frontend: `INSTANCE_ID=i-0172012226e3d165f`, `DOCUMENT=netpay-frontend-deploy` y `REPOSITORY=netpay-frontend`. La primera consulta puede responder `InvocationDoesNotExist` mientras SSM registra la ejecución; esperar y repetir con el mismo ID. Repetir hasta un estado terminal (`Success`, `Failed`, `Cancelled` o `TimedOut`); `Pending`, `InProgress` o `Delayed` no significan éxito. Si no finaliza, inspeccionar logs; no lanzar comandos de despliegue paralelos sobre el mismo componente.

Confirmar la imagen realmente usada sin imprimir las variables del contenedor:

sudo docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'

El tag guardado en `deploy.env` expresa la versión solicitada; contrastarlo con el contenedor activo y el resultado de SSM, especialmente después de una recuperación automática fallida.

Después, comprobar endpoint, UI, login y operación representativa. Registrar imagen/digest, comando y resultado. Corregir la rama de producción con un revert o una solución hacia adelante para que el siguiente despliegue no reinstale el fallo.

5. Árbol de resolución de incidentes

Síntoma	**Comprobación**	**Tratamiento**
Conexión rechazada o timeout	Estado EC2, EIP asociada, SG 80/443, ruta a Internet	Restablecer recurso o conectividad; después revisar contenedores
`ERR_SSL_PROTOCOL_ERROR`	Certificado de IP, `default_sni`, logs Caddy	Validar configuración y recargar; no desactivar TLS
Certificado vencido	Hora del host, acceso ACME, Caddy activo y volumen de certificados	Restablecer conectividad y permitir renovación; verificar desde fuera
Caddy devuelve 502	API healthy y red Compose	Diagnosticar arranque Spring/Flyway/SQL antes de reiniciar proxy
`/health` devuelve 503	`pg_isready`, logs PostgreSQL, disco y conexión JDBC	Resolver dependencia SQL; preservar volumen

Síntoma	Comprobación	Tratamiento
401	Token presente, firma, vencimiento, `token_credential`, estado usuario	Nuevo login o revisión de sesión; no ampliar permisos para resolver autenticación
401 `invalid_two_factor_code`	Desafío vigente, cinco intentos, consumo y cambios de cuenta	Repetir login y usar código nuevo; conservar ceros iniciales
400 `two_factor_retry_later`	Última emisión de código del usuario	Esperar al menos 60 segundos; no repetir envíos en bucle
Login con 2FA falla al enviar	Gmail OAuth2, remitente y logs depurados	Resolver envío; no se emite JWT sin segundo paso; `/health` puede seguir UP
403	Permiso efectivo o CORS	Distinguir respuesta `permission_denied` de rechazo de origen; revisar asignaciones vigentes
CORS desde navegador	Origen exacto, OPTIONS, cabeceras	Actualizar SecureString y reiniciar API; no usar `*` con credenciales
SSM Offline	Agente, rol de instancia y salida HTTPS	Revisar administración de EC2; no abrir SSH público como sustitución automática
`AccessDenied` en Actions	Usuario, secret, región, ARN exacto y política del documento/instancia	Corregir mínimo permiso necesario; una denegación explícita superior no se anula con otra política Allow
Imagen inexistente	Tag SHA y nombre ECR	Publicar/seleccionar imagen correcta; no inventar un tag mutable
Tag inmutable existente en backend	Reintento de publicación del mismo SHA	Redesplegar por SSM o crear nuevo commit; no sobrescribir la imagen

Síntoma	Comprobación	Tratamiento
Flyway checksum inválido	Historial aplicado y archivo modificado	Recuperar el script original y agregar migración nueva; no borrar historial
Falta `netpay-release`	cloud-init y `/var/log/cloud-init-output.log`	Esperar bootstrap o corregir el error de instalación; AL2023 trae `curl-minimal`
Correo o carga fallida	Credenciales y permisos del proveedor externo	Probar Gmail/Cloudinary de manera específica; `/health` no los cubre
CSV o `.xlsx` de reporte devuelve 400	`unsupported_report_format`, valor `format` enviado por la pantalla	API actual admite `xls` o `pdf`; XLS es binario real. Alinear versión/selector frontend, no renombrar archivos
Nuevo JSON de Parameter Store impide arrancar	Claves admitidas por `netpay-write-env` y validaciones de URL/longitud	Corregir esquema del parámetro; no exponer su valor en logs

6. Verificación de TLS y CORS

```
openssl s_client -connect 18.226.182.232:443 -noservername \
  -verify_return_error -verify_ip 18.226.182.232 -brief </dev/null

curl -i -X OPTIONS https://18.226.182.232/api/auth/login \
  -H 'Origin: https://3.143.92.7' \
  -H 'Access-Control-Request-Method: POST' \
  -H 'Access-Control-Request-Headers: content-type,authorization'
```

El primer comando prueba una conexión sin SNI y mantiene validación de certificado/IP. El segundo debe devolver el origen exacto permitido. Una prueba OpenSSL que envía manualmente SNI no basta para acreditar compatibilidad con un navegador que abre una IP. Mantener también una prueba normal con `curl` y con navegador.

Si cambia la configuración Caddy existente, validar y recargar dentro del contenedor. Reemplazar un archivo montado por bind mount puede conservar el inode anterior dentro del contenedor; comprobar el contenido visible o recrear solo el proxy para asegurar la nueva configuración. No recrear la base por una modificación TLS.

7. Respaldo y recuperación de datos

7.1 Estado de protección

PostgreSQL tiene un volumen Docker sobre EBS cifrado. No se declara un bucket externo de respaldos, un calendario de `pg_dump`, un servidor de copias incrementales ni archivado continuo de WAL. No se acredita PITR ni una restauración medida del ambiente productivo.

La política del proyecto solicita copia completa semanal, incremental diaria y log transaccional cada 15–30 minutos. Esa política es un objetivo de continuidad que requiere infraestructura adicional; no debe describirse como ejecutada por la configuración actual. El volumen EBS retenido y las imágenes ECR no sustituyen una copia recuperable de la base. V21 añade preferencia 2FA y desafíos de login: verificar si la copia y la imagen elegidas pertenecen a un esquema anterior o posterior a ese cambio.

7.2 Copia lógica manual

El siguiente procedimiento se ejecuta dentro de la EC2 backend con permisos de operación. Produce una copia local consistente; debe completarse con almacenamiento externo cifrado antes de considerarla un respaldo ante pérdida de instancia.

```
sudo install -d -m 700 /var/lib/netpay-backups

STAMP=$(date -u +%Y%m%dT%H%M%SZ)

BACKUP="/var/lib/netpay-backups/netpay-$STAMP.dump"

sudo sh -c 'umask 077; docker exec netpay-db-1 pg_dump -U netpay -d netpay_db -Fc > "$1"' sh "$BACKUP"

sudo test -s "$BACKUP"

sudo sha256sum "$BACKUP"

sudo sh -c 'docker exec -i netpay-db-1 pg_restore --list < "$1"' sh "$BACKUP"
```

Guardar fecha UTC, versión PostgreSQL, versión Flyway, SHA de aplicación, tamaño y checksum junto con el respaldo, sin datos secretos. El rol actual de EC2 no incluye escritura S3: cualquier automatización de copia externa requiere un destino aprobado y permisos restringidos para ese prefijo.

Una copia de `pg_dump` no es una copia incremental física ni incluye automáticamente archivos externos de Cloudinary. Un plan integral debe cubrir además esos activos, los secretos necesarios para recuperación y el estado de Terraform.

7.3 Prueba de restauración aislada

En un host de recuperación con Docker, PostgreSQL 16 y espacio suficiente, copiar el respaldo mediante un canal autorizado y verificar su checksum. Crear una base aislada sin puertos públicos:

```
read -r -s -p 'Contraseña temporal para el entorno de restauración: ' POSTGRES_PASSWORD

export POSTGRES_PASSWORD

docker run -d --name netpay-restore-check --network none \

  -e POSTGRES_PASSWORD -e POSTGRES_DB=netpay_restore postgres:16-alpine

unset POSTGRES_PASSWORD

docker exec netpay-restore-check pg_isready -U postgres -d netpay_restore
```

Esperar disponibilidad antes de restaurar; luego, con `BACKUP` apuntando al archivo copiado:

```
docker exec -i netpay-restore-check pg_restore -U postgres -d netpay_restore \

  --no-owner --no-acl --exit-on-error < "$BACKUP"

docker exec netpay-restore-check psql -U postgres -d netpay_restore \

  -c 'SELECT version, description, success FROM flyway_schema_history ORDER BY installed_rank;'
```

Conciliar cantidades y totales financieros, verificar referencias de recibos y ejecutar una versión compatible de la API contra esa copia aislada. Registrar tiempo de restauración y fecha del último movimiento recuperado. El catálogo de un dump o una ejecución sin errores no prueba por sí solos la validez de los saldos.

El contenedor de comprobación usa `--network none`: sirve para verificar SQL dentro del contenedor, pero no admite conexión desde otro contenedor API. Para la prueba funcional posterior, preparar una red Docker privada aislada y conectar únicamente la copia y la API de prueba, con configuración propia y sin puertos públicos ni credenciales de proveedores productivos. El arranque de esa API puede aplicar migraciones sobre la copia: registrar el

estado anterior y posterior. Eliminar el contenedor temporal y sus volúmenes de prueba solo después de guardar la evidencia y confirmar que no son recursos productivos.

7.4 Restauración productiva

1. Declarar incidente y suspender escrituras; impedir nuevas operaciones de API mientras se concilia.
2. Conservar la base dañada y sus evidencias. No sobrescribir la única copia existente.
3. Seleccionar un respaldo externo con checksum y prueba de restauración satisfactorios.
4. Restaurar en una base/volumen nuevo; comprobar versión Flyway y compatibilidad de imagen.
5. Recuperar archivos externos y configuración por los procedimientos del proveedor.
6. Reorientar la API, comprobar login, saldos, reportes y recibos con operaciones controladas.
7. Autorizar reapertura y registrar datos eventualmente no recuperados, RPO/RTO observados y acción preventiva.

Si solo se perdió EC2 pero sobrevive el EBS backend, la recuperación implica identificar y conectar el volumen correcto mediante una operación explícita o extraer su información. La declaración Terraform de una nueva EC2 crea otro disco raíz: no garantiza adopción del volumen retenido.

8. Operaciones que no son recuperación

No utilizar `docker compose down -v`, eliminación de EBS, `terraform destroy`, `flyway clean` o borrado de `flyway_schema_history` para reparar un error de arranque. No borrar pagos para que un reporte coincida con un total esperado. No aplicar una migración inversa sin análisis de datos y compatibilidad.

9. Cierre y registro del incidente

Registrar identificador del incidente, componente, impacto, hora UTC de detección y recuperación, versión anterior/nueva, comandos SSM, diagnóstico, intervención, validaciones y responsable. Adjuntar solo logs depurados de secretos y datos personales. Identificar si hubo pérdida de movimientos, si se necesita conciliación y qué control prevendrá la recurrencia.

Una intervención se cierra cuando la disponibilidad pública, los controles de acceso y la operación afectada están verificados. La mera finalización del comando remoto no demuestra integridad funcional.